TPG: informe

Software embarcado de una nave interestelar tripulada: Entrega I

Cátedra: Programación III

- Leonel Domingo Guccione
- Lucas De Lellis
- Micaela Rasso

Integrantes

- Andoni Catalano
- Sofía Ermelinda Casais
- Tomás Mastakas
- Eugenio Mateo Longaretto

Contenidos

- ❖ Objetivo
- ❖ Estructura principal
- ❖ Diagrama UML
- ❖ Patrones utilizados
- ❖ Algunas funciones clave
- ❖ Casos simulados

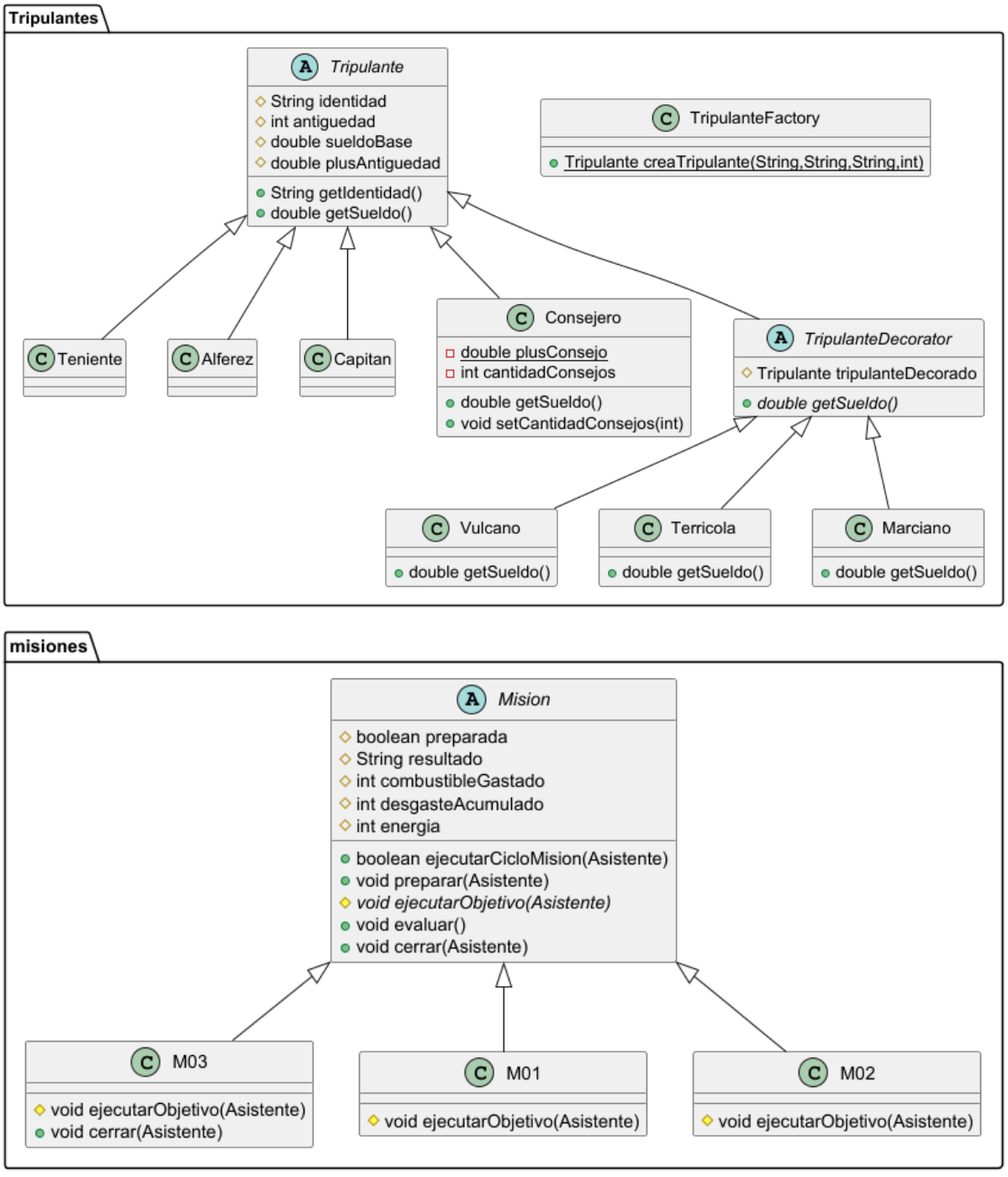
Objetivo

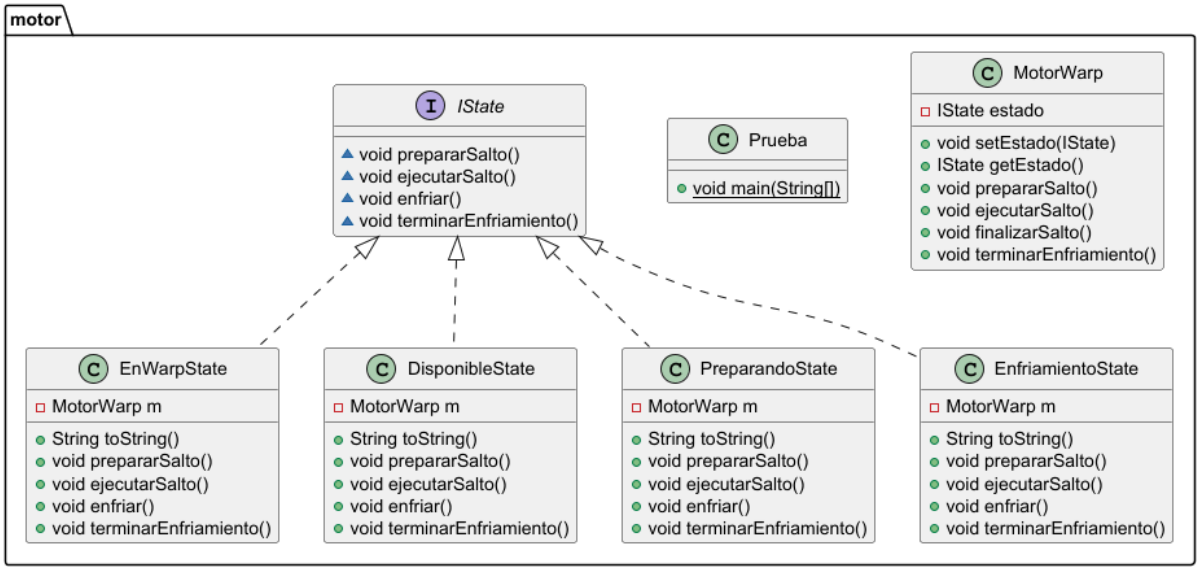
Este sistema fue desarrollado con el fin de poder gestionar naves espaciales y asignarle misiones que deben completar. Junto con ello debimos tener en cuenta los componentes internos de cada nave; entre ellos los tripulantes, el asistente y el motor

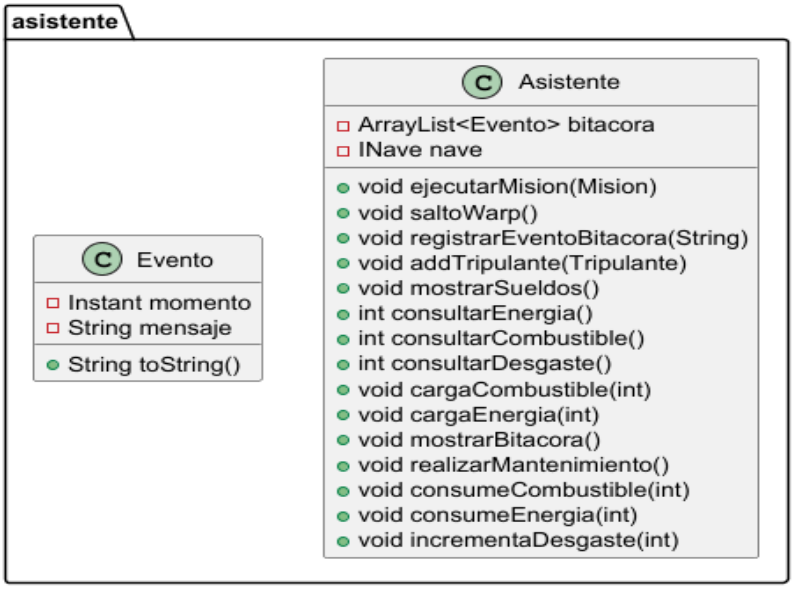
Estructura principal

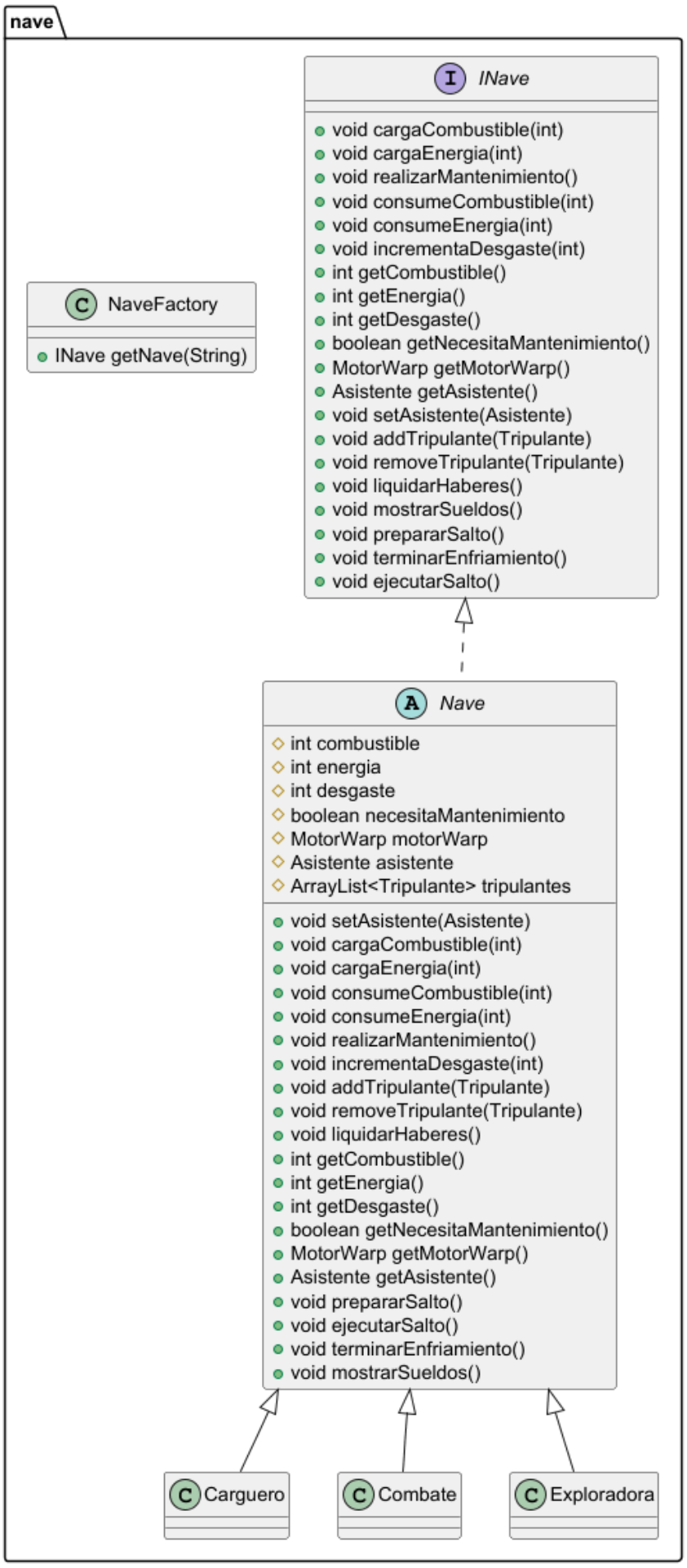
Sistema: Clase encargada de gestionar las naves del programa y crear misiones para luego encomendarselas a un asistente.
Asistente: Clase encargada de ejecutar misiones y coordinar las acciones que se quieren realizar sobre la nave.
Nave: Paquete encargado de la creación de las naves y de la ejecución de métodos necesarios para su funcionamiento
Tripulantes: Paquete encargado de crear tripulantes combinando su rol y origen. Este paquete también se encarga de calcular su sueldo según sea su rol, origen y antigüedad
Motor: Paquete que contiene, el motor, sus distintos estados, la interfaz de los estados y la excepción que evita transiciones inválidas. El motor tiene su estado y sus métodos que le permiten que cambie el propio estado.
Misiones: Paquete que contiene el procedimiento de las misiones junto a sus respectivos procesos.

Diagrama UML




motor
IState
void prepararSalto()
void ejecutarSalto()
void enfriar()
void terminarEnfriamiento()
Prueba
void main(String[])
MotorWarp
IState estado
void setEstado(IState)
IState getEstado()
void prepararSalto()
void ejecutarSalto()
void finalizarSalto()
void terminarEnfriamiento()
EnWarpState
MotorWarp m
String toString()
void prepararSalto()
void ejecutarSalto()
void enfriar()
void terminarEnfriamiento()
DisponibleState
MotorWarp m
String toString()
void prepararSalto()
void ejecutarSalto()
void enfriar()
void terminarEnfriamiento()
PreparandoState
MotorWarp m
String toString()
void prepararSalto()
void ejecutarSalto()
void enfriar()
void terminarEnfriamiento()
EnfriamientoState
MotorWarp m
String toString()
void prepararSalto()
void ejecutarSalto()
void enfriar()
void terminarEnfriamiento()


asistente
Asistente
ArrayList<Evento> bitacora
INave nave
void ejecutarMision(Mision)
void saltoWarp()
void registrarEventoBitacora(String)
void addTripulante(Tripulante)
void mostrarSueldos()
int consultarEnergia()
int consultarCombustible()
int consultarDesgaste()
void cargaCombustible(int)
void cargaEnergia(int)
void mostrarBitacora()
void realizarMantenimiento()
void consumeCombustible(int)
void consumeEnergia(int)
void incrementaDesgaste(int)
Evento
Instant momento
String mensaje
String toString()


nave
INave
void cargaCombustible(int)
void cargaEnergia(int)
void realizarMantenimiento()
void consumeCombustible(int)
void consumeEnergia(int)
void incrementaDesgaste(int)
int getCombustible()
int getEnergia()
int getDesgaste()
boolean getNecesitaMantenimiento()
MotorWarp getMotorWarp()
Asistente getAsistente()
void setAsistente(Asistente)
void addTripulante(Tripulante)
void removeTripulante(Tripulante)
void liquidarHaberes()
void mostrarSueldos()
void prepararSalto()
void terminarEnfriamiento()
void ejecutarSalto()
NaveFactory
INave getNave(String)
Nave
int combustible
int energia
int desgaste
boolean necesitaMantenimiento
MotorWarp motorWarp
Asistente asistente
ArrayList<Tripulante> tripulantes
void setAsistente(Asistente)
void cargaCombustible(int)
void cargaEnergia(int)
void consumeCombustible(int)
void consumeEnergia(int)
void realizarMantenimiento()
void incrementaDesgaste(int)
void addTripulante(Tripulante)
void removeTripulante(Tripulante)
void liquidarHaberes()
int getCombustible()
int getEnergia()
int getDesgaste()
boolean getNecesitaMantenimiento()
MotorWarp getMotorWarp()
Asistente getAsistente()
void prepararSalto()
void ejecutarSalto()
void terminarEnfriamiento()
void mostrarSueldos()
Carguero
Combate
Exploradora

Patrones utilizados

Factory: Este patrón fue utilizado en dos paquetes para evitar que el usuario conociese las clases concretas, por un lado se utilizó para crear la nave pasándole como atributo String el tipo de la nave. En el paquete tripulantes se utilizó para evitar que un tripulante pudiese acumular orígenes.
Template Method: Este patrón se utilizó como base para las misiones(M01, M02, M03).
State: Este patrón se usó para hacer las transiciones de estado que realiza el motor cuando quiere hacer un salto.
Decorator: Este patrón fue utilizado en los tripulantes, particularmente para el origen de los mismos.
Singleton: Este patrón fue utilizado en la clase Sistema, para que no sea posible la existencia de más de un Sistema a la vez.

Algunas funciones claves:

Incorporación de Nave al Sistema

El sistema incorpora una nueva nave a su flota invocando el método addNave(tipo), el cual delega la responsabilidad de instanciación a NaveFactory.getNave(tipo). El cual evalúa el parámetro y crea la nave concreta correspondiente, además de instanciar su Asistente para establecer de inmediato la vinculación bidireccional entre ambos objetos. Finalmente, el sistema recibe la nave y la añade a su colección, salvo que el tipo provisto sea desconocido, en cuyo caso se interrumpe el proceso arrojando una TipoNaveInvalidoException.

Fabricación de Tripulante

La creación e inicialización de la tripulación se centraliza mediante el método estático creaTripulante de la clase TripulanteFactory. Tras validar las precondiciones, se instancia el objeto base correspondiente a la jerarquía (Capitán, Alférez, Consejero o Teniente) y lo envuelve según el origen solicitado (Terrícola, Marciano o Vulcano). Luego, retorna la entidad lista para operar, o aborta la operación lanzando excepciones específicas.

Ejecución de Misión

El ciclo de una misión inicia cuando el sistema invoca ejecutarMision(nave, mision), extrayendo el Asistente vinculado a dicha nave para delegarle el control. El asistente inicia el proceso mediante mision.ejecutarCicloMision(this) y para luego evaluar su retorno. En caso de que el ciclo se corrompa por fallos lógicos de dominio, el asistente captura la ContratoMisionException y registra en la bitácora el mensaje de misión abortada.

Registro de Evento en Bitácora

Para registrar los eventos que suceden durante la ejecución, se invoca el método registrarEventoBitacora(mensaje) del Asistente. Tras verificar que la cadena de texto del

mensaje ingresado por parámetro no sea nula, se crea una nueva instancia de la clase Evento. Finalmente, el asistente añade este objeto consolidado a la bitácora para futuras consultas.

Casos simulados:

Escenario A: Ejecución Correcta

Se instancian tres naves mediante la fábrica. Posteriormente se selecciona una de ellas (la exploradora), asignándole una tripulación válida de cinco miembros y delegando al Asistente de Comando la ejecución secuencial de un ciclo de tres misiones (M-01, M-02 y M-03). Su objetivo es recorrer el camino ideal para demostrar que la creación de objetos, el patrón Template Method de las misiones y la gestión de recursos interactúan correctamente.

Salida por consola:

```
…
--- Recursos Finales ---
Combustible: 48
Energía: 90
Desgaste: 12

--- Bitácora Final ---
Evento:Mision preparada con exito
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:[M-01] Ejecutando objetivo
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:[M-01] Recolectando datos
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:[M-01] Mision ejecutada
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:Mision ejecutada: M01
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:Resultado de la mision: FRACASO
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:Energia agregada: 5
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:Combustible gastado: 4
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:Desgaste consumado: 4
Momento del Evento: 2026-10-11T16:26:03.609Z
Evento:Mision cerrada
Momento del Evento: 2026-10-11T16:26:03.609Z
…
```

La salida por consola confirma que las misiones completaron sus ciclos, registrando cada evento en la bitácora. Aunque el resultado aleatorio de los objetivos arrojó

"FRACASO" para las tres expediciones, el sistema procesó correctamente las consecuencias: los recursos fueron modificados (el combustible quedó en 48, la energía en 90 y el desgaste acumuló 12 puntos). Esto expone el cumplimiento de las respectivas postcondiciones.

Escenario B: Recursos Insuficientes

Se evalúa la reacción del sistema ante la falta de suministros instanciando una Carguero a la que se le vacía el tanque hasta dejarlo con solo 2 unidades de combustible. Posteriormente, se le ordena al Asistente de Comando iniciar la Misión 01 para comprobar si los contratos y las validaciones bloquean la operación antes de ejecutar cualquier descuento que deje a la nave con recursos negativos.

Salida por consola:

```
…
Combustible actual: 2
Intentando ejecutar Mision 01…

--- Bitácora del Rechazo ---
Evento: Fallo al preparar la misión: Recursos insuficientes o desgaste crítico
Momento del Evento: 2026-10-11T17:03:05.783Z

Verificación: El combustible sigue en 2 (No hubo cambios parciales).
```

El resultado demuestra que al detectar que el combustible no alcanza para el viaje, el sistema aborta la preparación automáticamente y registra el motivo del fallo en la bitácora ("Recursos insuficientes o desgaste crítico"). La verificación final confirma que el combustible se mantuvo en 2, probando que la operación se rechazó sin generar cambios parciales ni requerir que el programa crashee con excepciones.

Escenario C: Motor Warp

Se testea el funcionamiento del motor. El objetivo es forzar a la nave a recorrer un ciclo completo de transiciones válidas (preparar, ejecutar, finalizar y enfriar) y culminar con un intento de salto inválido envuelto en un try-catch para evaluar cómo el motor rechaza órdenes incompatibles con su fase actual.

Salida por consola:

```
…
Estado inicial: Disponible
Tras preparar: Preparando
Tras ejecutar: En Warp
Enfriando: Disponible

Exception in thread “main” motor.TransiciónInvalidaException: ERROR: el motor no necesita enfriarse
…
```

Por el momento no se tiene en cuenta el paso del tiempo, de modo que actualmente es imposible completar el camino completo (Disponible -> Preparando Salto -> En Warp -> Enfriamiento -> Disponible). Es por esto que, al correr el Main de prueba de este escenario, la consola expone que al ejecutar finalizarSalto(), el motor pasa de estar "En Warp" directamente a "Disponible" en lugar de pasar a "Enfriamiento". Por esto, cuando el código intenta realizar la siguiente transición legal (terminarEnfriamiento()), el programa aborta lanzando una TransicionInvalidaException indicando que "el motor no necesita enfriarse". Más allá de que hasta la primer entrega, es lo que debe pasar, esto demuestra que el patrón State funciona, bloqueando cualquier acción que no pertenezca a las reglas del estado vigente.

Escenario D: Contrato Inválido

Se instancia una nave de Combate (cuyo tanque de energía inicial arranca lleno en 100) y se le intenta cargar 50 unidades de combustible extra, de modo tal que exceda el valor límite (100). La operación se envuelve en un bloque try-catch para atrapar el error y aislar la consulta posterior de los recursos.

Salida por consola:

```
…
Energía inicial: 100
Intentando cargar 50 unidades de energía (el máximo es 100)...
Operación denegada: La carga de energía supera la capacidad máxima (100) o es inválida.
Energía final tras el rechazo: 100

¡Comprobado! El estado anterior se conservó perfectamente y no quedó en 150.
…
```

La impresión por consola muestra como el sistema bloquea el intento de sobrecarga arrojando la OperacionRechazadaException con su respectivo mensaje y, al consultar la nave, la energía se mantiene intacta en 100. Queda comprobado que ante la violación de una precondición, la nave aborta la transacción inmediatamente, conservando su estado original sin corromper los invariantes de clase.